BCL1213 Operating Systems

Tutorial 2: Process Management

Student Name: Chan Jing Yi
Student ID: SUOL2500321
Date: 21 October 2026

Tutorial Answers

Part A - Short Answer

1. Differentiate between a program and a process with examples.

A **program** is a passive set of instructions stored on a storage device. A **process** is an executing instance of a program, with its own execution state and allocated resources, including memory, a program counter and CPU registers.

For example, a calculator executable stored on a laptop is a program. Launching it creates a calculator process. If the application supports separate instances, launching it twice can create two processes from the same program, each with its own data and execution state.

2. List and explain four states in a process lifecycle.

Four valid states are shown below.

State	Explanation and example
New / Start	The OS is creating the process and setting up its resources; for example, when a calculator is launched.
Ready	The process can execute but is waiting for CPU time; for example, a calculation waiting in the ready queue.
Running	The process is currently executing instructions on a CPU; for example, performing the calculation.
Waiting / Blocked	The process cannot proceed until an event occurs; for example, a disk read completes.

The usual five-state model also includes **Terminated**, meaning execution has finished or the OS has ended the process. A ready process needs CPU time, whereas a blocked process needs an event or resource before it becomes ready again.

3. What information is stored inside a Process Control Block (PCB)?

A **PCB** is an OS data structure that records the information needed to manage a process and resume its execution.

Information	Purpose
Process ID (PID) and process state	Identify the process and record whether it is ready, running or waiting.
Program counter and CPU registers	Preserve where execution should resume and the saved processor values, including the stack pointer.
Scheduling information	Record priority, scheduling parameters and queue links.
Memory-management information	Identify the address space through page-table references or other memory mappings.
I/O status and open-file information	Track open files, assigned devices and outstanding I/O.
Accounting information	Record CPU usage, resource limits and ownership or account details.

4. Why is thread context switching faster than process context switching?

Switching between **threads in the same process** is generally faster because they share an address space and resources. The system saves and restores thread-specific state, such as registers, the program counter and the stack pointer, without normally changing the address space. A switch between processes also changes the address-space context and may create extra memory-translation overhead.

Both switches still have a cost. Kernel-managed thread switches involve the OS scheduler; only some user-level thread switches can occur without entering the kernel. Switching between threads belonging to different processes also requires changing address spaces.

Part B - Comparison

5. Compare processes and threads.

Aspect	Processes	Threads within the same process
Memory usage	Usually higher overhead: each process has its own virtual address space and management structures. Processes can explicitly share memory.	Usually lower overhead: threads share code, global data, heap and open files, but each has its own stack and execution state.
Creation speed	Generally slower because the OS must establish a process and its address space.	Generally faster because the existing process resources are reused.
Communication	Usually uses IPC, such as pipes, sockets or explicitly shared memory.	Can communicate directly through shared variables, with synchronization when needed.
Context switching	Usually more costly because the address-space context changes.	Usually less costly when switching within the same process.

These are typical comparisons; actual costs depend on the OS and workload.

6. Compare advantages and disadvantages of concurrency.

Concurrency means that multiple tasks make progress during overlapping periods. A single CPU core can interleave them; parallelism means executing tasks at the same instant on different execution units.

Advantages	Disadvantages
Responsiveness: an interface can accept input while background work continues.	**Race conditions:** unprotected shared data can produce incorrect results.
Resource use: the CPU can run another ready task while one waits for I/O.	**Deadlock or starvation:** tasks may wait indefinitely for resources or execution time.
Throughput: overlapping work can complete more tasks in a given period.	**Overhead and complexity:** scheduling, synchronization and debugging require extra work.
Multicore use: independent tasks may execute in parallel when cores are available.	**Contention:** tasks competing for locks, memory or devices can slow one another down.

Concurrency does not guarantee faster execution; its benefit depends on how much useful work can overlap and how much coordination is required.

Part C - Application / Scenario

7. Streaming YouTube while opening Facebook: which parts use processes and threads?

Processes: Assuming both websites are open in Chrome, the browser process manages the application interface and coordinates other processes. YouTube and Facebook content will normally run in separate renderer processes because they belong to different sites. Network and GPU services also operate in separate processes. This division provides isolation between components.

Threads: Each renderer contains a main thread for page work, including JavaScript and layout, together with other threads for tasks such as compositing and rasterization. Worker threads may execute background JavaScript. YouTube playback can involve media decoding and audio work on additional threads or services, while Facebook handles input and updates its page.

Thus, processes isolate browser components, while threads divide execution within those processes. On one core, runnable threads take turns; on multiple cores, some can run in parallel. The exact allocation depends on the browser, media implementation and site-isolation policy. A tab is not guaranteed to correspond to exactly one process.

8. Two bank threads update the same account balance: what can happen, and how does synchronization solve it?

A **race condition** can cause a **lost update** when both threads perform an unprotected read-modify-write operation. Suppose the initial balance is **RM1,000**, Thread A deposits **RM100**, and Thread B withdraws **RM200**.

Step	Thread A	Thread B	Shared balance
1	Reads RM1,000		RM1,000
2		Reads RM1,000	RM1,000
3	Calculates and writes RM1,100		RM1,100
4		Calculates from its old value and writes RM800	RM800

The correct balance is **RM1,000 + RM100 - RM200 = RM900**. The final RM800 value has lost the deposit. A different execution order could lose the withdrawal instead.

A **mutex lock** makes the complete update a critical section: a thread acquires the account's lock, reads the balance, checks any required condition, updates the balance, and releases the lock. Every thread accessing that balance must follow the same locking rule. The second thread then reads the first thread's completed update, producing RM900 in either serial order.

9. Why does a web browser use processes for tabs but threads inside a tab? Explain with examples.

Processes provide isolation. Separate renderer processes give websites separate address spaces and support sandboxing. For example, if a YouTube renderer crashes, Facebook content in another renderer process can usually keep working. Site isolation also helps separate data belonging to different sites. This protection has a memory and communication cost because the browser must maintain multiple processes and coordinate them through IPC.

Threads support cooperation within a renderer. Tasks for the same page often need access to related resources. For example, a renderer's main thread executes page JavaScript and performs layout, while a compositor thread can handle eligible scrolling and compositing work. This division helps keep the page responsive. Threads share their process's address space, reducing the need to send every piece of data through IPC, although shared data still requires coordination.

The statement “one process per tab” is a useful introductory simplification. Chrome's actual process assignment follows site and isolation requirements: some related tabs may share a renderer, while cross-site frames in one tab may use separate renderer processes. Processes supply the isolation boundary; threads supply execution paths within each process.